

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY (AUST)

Department of Computer Science and Engineering • Fall 2024 / Spring 2025

CSE 4108: Computer Graphics & Multimedia Laboratory

CASTLE ESCAPE

A 2D Puzzle-Platformer Adventure Game Using C/C++ & iGraphics

COMPREHENSIVE INDIVIDUAL PRESENTATION DEFENSE SCRIPTS

Slide-by-Slide Verbatim Dialogue, Visual Interaction Cues, Timecode Sync & Technical Defense Q&A

Team Presentation Roster & Module Work Distribution

Member Name	Student ID	Approved Proposal Domain	Assigned Code Modules	Allocated Slides	Speaking Slot
Ovijit Sharma LEAD	00725105101134	Game Theory, Aesthetics & Graphics, 60 FPS Physics, Player Controller & RenoSir Collab	CharacterRender.h RenderUtils.h GameLogic.h InputHandler.h	Slides 1 – 6	0:00 – 3:00 (3 Minutes)
Shahriar Rythm	00725105101132	Level Design Architect, 24-Floor Catalog, Interactive Traps, Dungeon Props & Solvers	Levels.h WorldObjects.h	Slides 7 – 10	3:00 – 6:00 (3 Minutes)
Nabeel Saad Borno	00725105101135	User Interface System, 3D World Carousel, Hero Select, Chapter Maps & Toast Pipeline	MenuScreens.h RenderUtils.h (UI)	Slides 11 – 14	6:00 – 9:00 (3 Minutes)
Maheed Abrar	00725105101140	Systems Architect, Binary Persistence, Audio Streaming, Daily Treasures & Penalty Derby	PenaltyGame.h SaveSystem.h AudioSystem.h	Slides 15 – 18	9:00 – 12:00 (3 Minutes)
All Members	Full Team	Joint Project Synthesis, Future Roadmap, Interactive Live Demonstration & Faculty Q&A	iMain.cpp demo.exe	Slide 19	12:00 – 15:00 (Wrap-up & Q&A)

Master 12-Minute Defense Schedule

Timecode	Phase	Active Presenter	Primary Presentation Focus	Key Deliverables Shown
0:00 – 0:45	Slide 1	Ovijit Sharma (Team)	Atmospheric Cold Open & Narrative Hook	Procedural stone dungeon, glowing prompt, diamond HUD
0:45 – 1:30	Slide 2	Ovijit Sharma (Team)	Project Scope & Team Work Distribution	Proposal compliance matrix, modular architecture, C++ lines
1:30 – 3:45	Slides 3–6	Ovijit Sharma	Physics, Chibi Animations & RenoSir Collab	14-state FSM, Euler integration, squash/stretch, RenoSir lore
3:45 – 6:45	Slides 7–10	Shahriar Rythm	Level Architecture, Traps & Tricky Puzzles	24-floor catalog, 3D red button, gravity pads, hint spawner
6:45 – 9:45	Slides 11–14	Nabeel Saad Borno	Vector UI, 3D Carousel & Star Rating Engine	Resolution-independent buttons, depth scaling, toast pipeline
9:45 – 12:45	Slides 15–18	Maheed Abrar	Persistence, MCI Audio & CR7 Penalty Derby	Binary save serialization, 7-day streak, Messi AI & test harness
12:45 – 15:00+	Slide 19	Joint Team	Conclusion, Future Scope & Live Gameplay Demo	F5 live demo in Visual Studio, puzzle playthrough & faculty Q&A
Project Repository: TorchboundKeep / CastleTrapVS2013			Platform: Native C/C++ with OpenGL & iGraphics (Win32)	Status: 100% Objectives Delivered

PRESENTER 01: OVIJIT SHARMA

Slides 1 - 6

Student ID: 00725105101134 • Lead Game Theorist, Art Director & Physics Architect Allocated Time: 0:00 - 3:00 (3 Mins)

SLIDE 1 Exact Game Intro & Poetic Cold Open

0:00 - 0:45 (45s)

VERBATIM SPOKEN SCRIPT

"Honorable course teachers and respected evaluation committee, good morning. We warmly welcome you to our Computer Graphics Lab project defense for **Castle Escape: A 2D Puzzle Adventure Game Using iGraphics.**"

"Rather than starting with an ordinary title slide, we open directly with the authentic cold open of our game, replicating line-for-line the poetic sequence in `iMain.cpp`. Built natively in C and C++ on OpenGL, this screen establishes our moody dungeon atmosphere: procedural stone-brick masonry, dark radial vignette shading, and our Chibi Hero standing alone."

"As the text reads: *'I came a long way to get here... there were dozens of traps, hundreds of puzzles on my way.'* This project represents a complete, modular, physics-driven puzzle-platformer designed to prove that mathematical algorithms and vector rendering can deliver a modern indie game. Let us enter the fortress."

SLIDE VISUALS

Stone masonry background, dark radial vignette, Chibi Hero avatar, diamond HUD buttons, pulsating gold prompt.

PRESENTER ACTIONS

ACTION Point to the glowing text and CRT vignette.

TRIGGER Press `Space` or click anywhere to transition to Slide 2.

SLIDE 2 Computer Graphics Project Defense — Roster & Scope Matrix

0:45 - 1:30 (45s)

VERBATIM SPOKEN SCRIPT

"On Slide 2, we present our team work distribution matrix, mapped 1-to-1 against our approved Computer Graphics Project Proposal. We have structured our presentation into four strict 3-minute technical segments:"

"First, I, **Ovijit Sharma**, led Game Theory, Procedural Aesthetics, 60 FPS Physics Simulation, and our RenoSir AI Collaboration across `CharacterRender.h`, `RenderUtils.h`, `GameLogic.h`, and `InputHandler.h`."

"Second, **Shahriar Rythm** engineered our 24-floor dungeon catalog, interactive pressure plates, and tricky puzzle mechanics in `Levels.h` and `WorldObjects.h`."

"Third, **Nabeel Saad Borno** created our resolution-independent 2D Vector UI, 3D world carousels, and toast pipeline in `MenuScreens.h`."

"Fourth, **Maheed Abrar** architected our binary file persistence, MCI audio engine, and standalone Penalty Shootout mini-game in `PenaltyGame.h`, `SaveSystem.h`, and `AudioSystem.h`."

"Every single feature proposed has been coded and verified in C++ with 100% compliance."

SLIDE VISUALS

4 color-coded member cards, IDs, assigned headers, line count metrics (~8,700 total lines), proposal scope tags.

PRESENTER ACTIONS

ACTION Acknowledge each teammate with an open hand gesture.

EMPHASIS Highlight strict modular separation of header files.

 VERBATIM SPOKEN SCRIPT

"Moving into my technical contributions: in GameLogic.h and GameDefines.h, I designed the core puzzle-platformer game theory and our 60 FPS deterministic physics engine."

"To eliminate race conditions and rogue input during screen transitions, I engineered an explicit **14-state Finite State Machine** (enum GameState), ensuring strict synchronization between menus, gameplay, cutscenes, and the penalty derby."

"Our physics loop runs at a fixed ~16.6ms timestep via iSetTimer(16, updateGame). We implement numerical **Euler Integration**: vertical velocity decrements by a gravity constant of 0.52 units per frame, clamped at a terminal velocity of 14 to prevent tunneling through tiles."

"Crucially, I engineered a **Dual-Gravity Tensor**: standard gravity pulls downward, but runic pads invert the gravitational polarity (gravityInverted = true), allowing inverted ceiling walking. Collision is resolved via swept Axis-Aligned Bounding Box (AABB) intersection tests against solid tiles."

 SLIDE VISUALS

14-state FSM diagram, Euler integration equations, collision code snippet, interactive gravity & jump simulator.

 PRESENTER ACTIONS

INTERACTIVE Click ⚡ Normal

Jump in the simulator.

INTERACTIVE Click 🔄 Invert

Gravity Jump to demonstrate ceiling walking live!

 VERBATIM SPOKEN SCRIPT

"Standard 2D games often rely on megabytes of static sprite-sheet bitmaps. In CharacterRender.h, I took an advanced algorithmic approach: engineering an organic **procedural deformation pipeline** in pure vector mathematics."

"Stationary heroes are brought to life through a sinusoidal idle breathing harmonic: $\sin(\text{levelTimer} * 0.045) * 0.028$, causing a subtle, organic chest expansion."

"When moving, our articulated run stride calculates vertical bobbing: $\text{bobY} = |\sin(\text{walkCycle} * 0.20)| * 4.0$, synchronized with audio footsteps."

"Furthermore, we implement classic Disney animation principles: at the jump apex, the sprite stretches vertically ($\text{scaleX} = 0.90$, $\text{scaleY} = 1.12$); upon floor impact, it squashes to $\text{scaleX} = 1.18$, $\text{scaleY} = 0.82$ before returning to equilibrium. Directional flips are handled seamlessly by negating coordinate offsets without texture artifacts."

 SLIDE VISUALS

Chibi deformation math, trigonometric wave diagrams, Interactive Chibi Animation Lab with state buttons.

 PRESENTER ACTIONS

DEMO Click 🌀 Idle Sway ,

then 🏃 Run Bounce .

DEMO Click 🦘 Jump Stretch

& 🦘 Land Squash .

 VERBATIM SPOKEN SCRIPT

"To create a distinct visual signature for Castle Escape, I developed procedural graphics and lighting routines in `RenderUtils.h` and `InputHandler.h`."

"Because `iGraphics` lacks built-in fragment shaders, I created `drawGlowingText()` using multi-pass concentric alpha blending. By rendering repeated text passes with expanding pixel offsets and exponentially decaying alpha channel intensities, we produce a dynamic pulsing bloom effect."

"For atmospheric depth, I implemented a full-screen dark vignette using `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`, casting shadowy corners while keeping our hero illuminated. We also generate metallic cards with beveled rims and a buoyant torch ember particle system."

"In `InputHandler.h`, I unified keyboard (WASD/Arrows/Space/E) and mouse click event dispatching into single-frame action triggers like `executeJump()` and `interactWorldObject()`."

 SLIDE VISUALS

Concentric alpha bloom math, OpenGL blend modes, metallic card render code, `InputHandler` dispatch routing.

 PRESENTER ACTIONS

POINT Highlight the ember particle effect floating on the presentation canvas.

CODE Explain the `glBlendFunc` parameters.

SLIDE 6 Special Feature Spotlight: RenoSir Collaboration

 VERBATIM SPOKEN SCRIPT

"Our final character innovation is our premier crossover collaboration champion: **RenoSir — LLM Researcher & AI Strategist** (`HERO_SAHA_RENO`)."

"We connected advanced artificial intelligence themes with dungeon platforming. In our game lore, RenoSir utilizes algorithmic reasoning and predictive LLM foresight to anticipate tricky traps. His stats reflect this agility: boasting top-tier Speed of 4.0 and Jump Power of 12.0, allowing players tight platforming margins across difficult spike chambers."

"I engineered an interactive **1-Level Trial System**: any player can test RenoSir immediately for one entire floor from the collaboration banner by toggling `renoSirOneLevelTrial = true`. RenoSir is balanced alongside Sir William, Red Devil CR7, Neymar Jr, Elena, and Thorgar. As Maheed will detail later, RenoSir is permanently unlocked as the Day 7 grand reward in our daily calendar."

 SLIDE VISUALS

RenoSir neural avatar, stat bars (Speed 4.0, Jump 12.0), 1-Level Trial modal preview, 6 Chibi heroes matrix.

 PRESENTER ACTIONS

EMPHASIS Highlight the crossover lore and 1-level trial mechanic.

PREVIEW Show the RenoSir collaboration modal card.

 Verbal Handover to Shahriar Rythm:

*"With the foundational game theory, physics engine, and character animation pipeline established, I now hand over the presentation to our Level Architect, **Shahriar Rythm**, to present our dungeon catalog, trap mechanics, and puzzle design."*

 Ovijit Sharma — Technical Defense Q&A Preparation**Q: Why did you choose fixed Euler Integration over Verlet or Runge-Kutta in `GameLogic.h`?**

A: In a discrete 60 FPS grid-based platformer, fixed-step Euler integration provides guaranteed determinism, low computational complexity, and instantaneous velocity dampening upon collision contact. Since timestep `dt` is clamped to ~16.6ms by `iSetTimer`, Euler integration exhibits no observable drift for 2D platforming.

Q: How does `drawGlowingText()` work without native shader support in `iGraphics`?

A: We emulate bloom by executing multiple concentric rendering passes of the string. Each outer pass is offset radially by `delta-r` with alpha attenuated quadratically, using OpenGL alpha blending (`glEnable(GL_BLEND); glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`).

PRESENTER 02: SHAHRIAR RYTHM

Student ID: 00725105101132 • Level Design Architect, Trap Mechanics & Dungeon Props

Slides 7 - 10

Allocated Time: 3:00 – 6:00 (3 Mins)

SLIDE 7 Level Design Architecture & 24-Floor Catalog

4:30 - 5:15 (Rythm 0:00 - 0:45)

VERBATIM SPOKEN SCRIPT

"Thank you, Oviijit. Respected teachers, I am **Shahriar Rythm**, and my core focus was level design, spatial puzzle geometry, and dungeon progression across `Levels.h` (comprising over 600 lines of C++)."

"Castle Escape features a comprehensive **24-floor catalog** spanning four distinct castle chapters, engineered with a deliberate multi-chapter difficulty curve:"

"**Chapter 1: Princess Castle** (Floors 1 to 12) introduces foundational puzzle platforming: ceiling pull-chains, decoy phantom chests, falling keys, and hidden push-doors."

"**Chapter 2: Royal Citadel** (Floors 13 to 16) introduces dynamic mechanical entities: gravity inversion pads, cogwheel winches, and spring jump pads."

"**Chapters 3 and 4** (Witch Tower and Dragon's Keep) introduce arcane teleportation portals, timed stone crushers, lava geysers, and rolling boulder traps."

"To manage this cleanly, I designed the `LevelInfo` struct, encapsulating chapter titles, puzzle hints, instant solutions, sarcastic death messages, spawn points, and door lock states."

SLIDE VISUALS

4-chapter progression breakdown, 24-floor catalog overview, `LevelInfo` struct architecture code box.

PRESENTER ACTIONS

POINT Walk through the 4 chapter banners on screen.

EXPLAIN Detail the fields of the `LevelInfo` struct.

SLIDE 8 Trap Mechanics & Interactive Dungeon Props

5:15 - 6:00 (Rythm 0:45 - 1:30)

VERBATIM SPOKEN SCRIPT

"In `WorldObjects.h`, I authored over 520 lines of code dedicated to transforming static level backgrounds into reactive, physics-driven puzzle spaces."

"Our signature puzzle prop is the **3D Red Floor Button** (`drawRedButton`). When stepped on by the player or weighted down by a heavy crate, the button physically depresses 8 pixels downward with a mechanical click, closing a circuit to open gates or extend drawbridges."

"I also developed procedural **Medieval Castle Torches** (`drawCastleTorch`), generating animated flickering flames via trigonometric particle dispersion without any pre-rendered video files."

"Furthermore, we have **Ceiling Pull-Chains** (`drawCeilingChain`). Players can leap and pull these chains, triggering physics callbacks that lower heavy timber drawbridges across bottomless spike pits. Our exit doors support wooden passage doors, iron-barred portcullises, and locked doors with glowing keyholes."

SLIDE VISUALS

3D red button rendering, medieval torch particles, ceiling chain, Interactive Trap Sandbox.

PRESENTER ACTIONS

INTERACTIVE Click  Pull Ceiling Chain [E] to lower drawbridge.

INTERACTIVE Click  Step on 3D Button to unlock exit door!

 VERBATIM SPOKEN SCRIPT

"In Chapter 2, puzzle mechanics advance from simple jumping into lateral puzzle-solving, drawing inspiration from *Tricky Castle*."

"First, we introduce **Gravity Reverse Pads**. Stepping onto these glowing runic tiles reverses the player's gravitational vector, causing them to fall upward and walk along the ceiling to reach high-altitude keys."

"Second, our **Wooden Spring Jump Pads** (`drawWoodSpringPad`) compress on impact and propel the hero 2.5 times higher than a standard jump, allowing access to secret lofts."

"Third, we engineered **Pushable Crates**: rigid physics bodies that can be shoved across platforms to hold down pressure plates or dropped into spike pits as stepping stones."

"Most importantly, our level design deliberately subverts conventional video game tropes. For example, in **Floor 4 — 'Use Force, Not Key'**, players search in vain for a key that does not exist; the solution is to shove the heavy wooden door open with brute force! In **Floor 5**, the key drops toward a fatal death trigger, requiring the player to sprint and catch the key in mid-air before it activates the ceiling crusher."

 SLIDE VISUALS

Gravity pad, spring launcher, pushable crate diagrams, "Tricky" expectation subversion callout cards.

 PRESENTER ACTIONS

EMPHASIS Contrast conventional game logic with Tricky Castle puzzle design.

EXPLAIN Walk through Floor 4 and Floor 5 puzzle solutions.

 VERBATIM SPOKEN SCRIPT

"A key systems achievement in `Levels.h` is our central room lifecycle engine, encapsulated in `loadLevel(int lvl)`."

"When entering a floor, `loadLevel()` cleanly resets and instantiates all dynamic entity arrays: spikes, levers, chests, falling stones, and moving platforms. Because we avoid dynamic heap allocations inside the render loop, restarting a floor upon death is instantaneous (zero load latency), eliminating player downtime."

"`loadLevel()` also preserves persistent state, such as verifying whether `RenoSir`'s 1-level trial is currently active."

"Finally, to maintain player accessibility without trivializing challenge, I engineered an **In-Game Hint & Solution Dispenser**. If a player is stuck, pressing the hint icon displays a subtle cryptic hint. If they have earned a 'Hint Pass' from daily treasures, they can reveal the exact solution walkthrough text."

 SLIDE VISUALS

Entity lifecycle flow, instant reset handler, `showRoomHintDialog` code box, hint scroll UI.

 PRESENTER ACTIONS

CODE Point to the two-tier hint and instant solution delivery mechanism.

EMPHASIS Zero heap fragmentation on level reset.

 Verbal Handover to Nabeel Saad Borno:

"Now, to demonstrate how these complex puzzle mechanics and floor catalogs are presented through a unified, high-relief user interface, I hand over the presentation to our UI/UX Lead, **Nabeel Saad Borno**."

Shahriar Rythm — Technical Defense Q&A Preparation

Q: How do you prevent memory leaks when loading and reloading 24 different levels in Levels.h?

A: All room entity collections (buttons, spikes, platforms) use fixed static arrays sized to maximum entity bounds (MAX_PLATFORMS, MAX_PROPS). loadLevel() re-initializes entity counts and properties in-place without dynamic malloc/free calls, completely eliminating heap fragmentation.

Q: How does the 3D red button determine if a player or crate is standing on it?

A: The button entity defines an active trigger AABB zone. During updateWorldObjects(), we check if the player's foot bounding box or any crate's bottom edge intersects the trigger zone. If true, button.depressed is set to true and y-offset shifts by 8 pixels downward.

PRESENTER 03: NABEEL SAAD BORNO

Slides 11 - 14

Student ID: 00725105101135 • User Interface System, 3D Menus & Graphics Elements Allocated Time: 6:00 – 9:00 (3 Mins)

SLIDE 11 User Interface Design System & High-Relief Vector HUD

7:30 - 8:15 (Nabeel 0:00 - 0:45)

VERBATIM SPOKEN SCRIPT

"Thank you, Shahriar. Respected faculty members, I am **Nabeel Saad Borno**, and my responsibility was designing and coding the entire user interface, menus, carousel mathematics, and HUD systems in MenuScreens.h (comprising 1,363 lines of C++)."

"Rather than relying on static, pixelated bitmap buttons that distort when scaled, I established a **resolution-independent 2D vector UI design system**."

"Our primary interaction element is draw3DButton(). Each button features a beveled top highlight rim, a gradient face, a darker cast underside, and a 2-pixel downward translation upon mouse hover and click, giving tactile physical feedback."

"Our in-game HUD provides immediate telemetry: the active floor banner, a diamond pause toggle, dynamic coin and gem counters, and an active hero badge showing whether a champion or a RenoSir 1-Level Trial is currently active."

"I established a curated color hierarchy: Warm Amber (#F59E0B) for interactive calls-to-action, Arcane Cyan (#38BDF8) for energy and trials, and Slate Navy for card backdrops."

SLIDE VISUALS

High-relief 3D button rendering, top in-game HUD banner, currency counters, RenoSir Trial HUD Pill.

PRESENTER ACTIONS

HOVER Hover cursor over the interactive 3D button on screen.

POINT Point to the live coin and gem counters and trial pill.

SLIDE 12 3D World Select & Hero Customization Carousel

8:15 - 9:00 (Nabeel 0:45 - 1:30)

VERBATIM SPOKEN SCRIPT

"To capture the authentic, premium look of *Tricky Castle*, I engineered a **pseudo-3D carousel navigation system** in drawWorldSelectMenu()."

"Using pure 2D graphics primitives, we simulate 3D cylindrical depth through mathematical perspective scaling. The central selected world card is rendered at 100% scale and full opacity, while neighboring cards to the left and right are mathematically scaled down to 82% size with opacity attenuated to 0.65."

"When navigating between worlds, we apply trigonometric ease-out interpolation, creating a smooth gliding transition across cards."

"I applied this identical pseudo-3D depth mathematics to our **Champion Selection Carousel**, which presents all 6 playable heroes: Sir William, CR7, Neymar Jr, Elena, Thorgar, and RenoSir. Each card renders dynamic stat bars for Speed, Jump Power, and Unique Abilities, updating in real time."

SLIDE VISUALS

Pseudo-3D depth carousel formulas, Z-axis scaling ratios (100% vs 82%), Interactive World Carousel Simulator.

PRESENTER ACTIONS

INTERACTIVE Click ◀ Previous World .

INTERACTIVE Click Next World ▶ to demonstrate live depth scaling!

VERBATIM SPOKEN SCRIPT

"Once a player selects a world, they enter our Chapter Map selection screen."

"Each chapter displays a 12-floor interactive navigation grid. Locked floors are rendered with a dimmed metallic shader and a lock icon."

"I developed our **Dynamic 3-Star Rating Engine**. Every completed floor awards between 1 and 3 stars based on three clear criteria:"

"First: completing the room; Second: beating the par completion time; and Third: completing the floor with zero deaths while collecting the hidden gold coin."

"Stars are rendered procedurally using `drawStarShape()`, which calculates the ten alternating vertices of a golden five-pointed star using polar coordinates. Accumulated stars unlock milestone rewards across the game. For example, earning 15 total stars automatically unlocks the exclusive *Juventus CR7* hero skin."

SLIDE VISUALS

12-floor chapter grid, locked state shader, procedural golden star polar coordinates, `drawFloorCard` code box.

PRESENTER ACTIONS

POINT Show the procedural star geometry generated without bitmap images.

CODE Explain the star rating evaluation logic.

VERBATIM SPOKEN SCRIPT

"Finally, I engineered the modal dialog and notification architecture."

"A major challenge in iGraphics is providing player feedback without freezing the rendering loop. To solve this, I built a **non-blocking toast notification queue** via `showToast()`. When triggered, toasts float upward from the bottom of the screen with an ease-out trajectory, remain fully opaque for 120 frames (~2 seconds), and gracefully fade out using alpha blending."

"I also designed the **RenoSir Collaboration Modal**, featuring custom banner artwork, lore text, and the 'Close & Enjoy 1-Level Trial' action button."

"Additionally, I authored our comprehensive Settings menu, providing discrete volume sliders for background music and sound effects, resolution diagnostics, and save reset options."

SLIDE VISUALS

Toast notification ease-in/out curve, RenoSir Collaboration modal card, Settings screen, Interactive Toast Simulator.

PRESENTER ACTIONS

INTERACTIVE Click  Key Toast .

INTERACTIVE Click  Trial Toast to fire live toasts on screen!

 Verbal Handover to Maheed Abrar:

"Now, to explain our systems architecture, binary file persistence, Windows audio engine, and our standalone CR7 vs Messi Penalty Derby mini-game, I invite our Systems Architect, **Maheed Abrar**."

 Nabeel Saad Borno — Technical Defense Q&A Preparation**Q: How do you achieve the 3D depth effect in the World Select carousel without a 3D perspective matrix?**

A: We use pseudo-3D trigonometric depth projection. Cards are positioned along an elliptical arc ($x = cx + r \cdot \sin(\theta)$). The scale factor is computed as $s = 0.82 + 0.18 \cdot \cos(\theta)$, and alpha opacity is modulated as $a = 0.65 + 0.35 \cdot \cos(\theta)$, sorting rendering order by depth.

Q: How do toast notifications operate asynchronously without blocking iGraphics execution?

A: We store toast state in a struct (message, frameCount, alpha). In `updateGame()`, frameCount increments every tick. The render loop evaluates $\alpha = \text{clamp}(\sin(\pi \cdot \text{frame} / 120), 0.0, 1.0)$ and draws the toast over the scene without halting game flow.

PRESENTER 04: MAHEED ABRAR

Student ID: 00725105101140 • Systems Architect, Persistence, Audio & Penalty Derby
Mini-Game Lead

Slides 15 - 18

Allocated Time: 9:00 - 12:00 (3 Mins)

SLIDE 15 Systems Architecture, Audio Engine & Persistence

10:30 - 11:15 (Maheed 0:00 - 0:45)

VERBATIM SPOKEN SCRIPT

"Thank you, Nabeel. Respected teachers, I am **Maheed Abrar**, and my focus was systems engineering, data persistence, the audio engine, and mini-game development across `SaveSystem.h`, `AudioSystem.h`, and `PenaltyGame.h`."

"First, in `SaveSystem.h`, I implemented a **binary file serialization architecture** targeting `tricky_castle_save.dat`. Rather than slow text parsing, we write binary structs atomically: recording all 24 floor unlock flags, 3-star records, coin and gem inventories, unlocked champions, and login streak counters in a single pass."

"To ensure stability, I built an automatic corruption fallback: if the file is missing or contains invalid byte counts, default profile values are instantiated, ensuring the game never crashes."

"Second, in `AudioSystem.h`, I engineered our audio engine leveraging the Windows Media Control Interface (`mciSendString`) via `winmm.lib`. This enables seamless, low-overhead streaming of high-quality looping MP3 and WAV background music alongside concurrent sound effects for footsteps, button clicks, and celebrations."

SLIDE VISUALS

Binary serialization schema, corruption fallback diagram, Windows MCI audio streaming code box.

PRESENTER ACTIONS

CODE Explain the `mciSendString` loop command.

EXPLAIN Detail the atomic struct read/write to prevent partial writes.

SLIDE 16 Daily Game Rewards & RenoSir Collab Integration

11:15 - 12:00 (Maheed 0:45 - 1:30)

VERBATIM SPOKEN SCRIPT

"A major gameplay innovation I introduced was connecting Ovijit's RenoSir collaboration character to long-term player retention via our **7-Day Login Treasures System**."

"Each day a player logs in, they advance along a 7-day milestone calendar: earning Gems on Day 1, Coins on Day 2, Master Keys on Day 3, and culminating on **Day 7 with the permanent unlock of RenoSir (AI Strategist)!**"

"However, we introduced a game theory twist: daily rewards are not claimed passively by simply clicking a button. To claim the reward, the player must step up and score a penalty against Messi in our **Penalty Shootout Derby!**"

"This creates thrilling risk-reward dynamics: If the player scores, the daily reward is granted, celebration confetti rains down, and CR7 performs his iconic SIUUU celebration! If the player misses or Messi saves it, the daily streak resets, Messi taunts the player, and they must re-attempt to earn their prize!"

SLIDE VISUALS

7-day calendar interface, Day 7 RenoSir unlock box, penalty claim mechanism, win vs miss feedback.

PRESENTER ACTIONS

POINT Highlight Day 7 in the calendar on screen.

EMPHASIS Explain the gameplay synergy between platforming and shootout.

VERBATIM SPOKEN SCRIPT

"In PenaltyGame.h, I authored nearly 1,600 lines of C++ to create a complete, standalone football shootout mini-game featuring Cristiano Ronaldo versus Lionel Messi."

"The shooting mechanics feature two core controls: First: a responsive crosshair tracking mouse and keyboard aiming coordinates; Second: an oscillating power gauge that charges from 0% to 100% when holding the spacebar."

"When kicked, we simulate a **3D parabolic trajectory**: the football's visual radius scales down dynamically from 24 pixels to 9 pixels as it travels toward the goal line, creating authentic 3D depth in a 2D engine."

"I also developed **Messi's Goalkeeper AI**. Messi analyzes the shot trajectory with a humanized reaction latency, deciding whether to make a diving stop, leap toward the top corner, or stay grounded. If a goal is scored, procedural harmonic displacement ripples across the goal net strings."

SLIDE VISUALS

Shootout arena, power gauge charging, 3D ball depth scaling (24px to 9px), Messi AI dive vectors, Interactive Shootout Arena.

PRESENTER ACTIONS

INTERACTIVE Click ▲ Top

Corner in the arena simulator!

DEMO Show Messi diving and the goal ripple reaction live!

VERBATIM SPOKEN SCRIPT

"To ensure rock-solid stability during grading and evaluation, I built an automated quality assurance and diagnostic test harness in iMain.cpp."

"By launching the executable with the --autocapture command-line argument, the game runs headlessly, programmatically traversing all 14 game states without requiring human input."

"To verify rendering correctness, I authored saveGLScreenshot(). This function reads the raw OpenGL backbuffer directly using glReadPixels(0, 0, width, height, GL_RGB, GL_UNSIGNED_BYTE, pixels) and encodes uncompressed 24-bit BMP image files to disk for automated visual regression auditing."

"We also performed rigorous memory audits, ensuring that switching between 24 levels and the penalty mini-game produces zero OpenGL texture leaks and zero heap fragmentation. I will now invite our entire team forward to conclude our presentation."

SLIDE VISUALS

Headless test harness flow, OpenGL backbuffer pixel extraction, BMP file encoder, memory audit metrics.

PRESENTER ACTIONS

CODE Explain the glReadPixels memory buffer allocation.

TRANSITION Signal the rest of the team to step forward to the podium.

Verbal Handover to Joint Team Conclusion:

"I now invite all members of our team forward to conclude our presentation and initiate our interactive live gameplay demonstration."

Maheed Abrar — Technical Defense Q&A Preparation

Q: Why did you write raw binary structs instead of JSON/XML in SaveSystem.h?

A: In C/C++ without external third-party parsing dependencies, fwrite and fread of a fixed-layout POD struct execute in sub-millisecond time (atomic disk write). It eliminates string parsing overhead and protects against save-file tampering.

Q: How does the 3D ball projectile achieve perspective depth in PenaltyGame.h?

A: The ball calculates an interpolation factor t from 0.0 (penalty spot) to 1.0 (goal line). Current radius is linearly interpolated: $r(t) = 24.0 * (1.0 - t) + 9.0 * t$. Vertical position follows a quadratic trajectory $y(t) = y_0 + v_y * t - 0.5 * g * t^2$, providing convincing 3D flight.

JOINT PRESENTATION CONCLUSION & LIVE DEMO

SLIDE 19

Slide 19 • Full Team Delivery (Ovijit, Shahriar, Nabeel, Maheed)

Wrap-up & Live Demo: 12:00 – 15:00

SLIDE 19 Project Synthesis, Future Roadmap & Live Demonstration

12:00 – 15:00+ (3 Mins)

VERBATIM SPOKEN SCRIPT

Ovijit Sharma: "Respected faculty members, in conclusion, *Castle Escape* stands as a complete, fully realized 2D puzzle-platformer. We have delivered 100% of the objectives outlined in our approved Computer Graphics Project Proposal:

"A deterministic 60 FPS physics engine, 24 fully populated puzzle floors across 4 castle chapters, 6 balanced heroes including our exclusive RenoSir AI collaboration, a standalone CR7 vs Messi penalty shootout, and robust binary persistence."

Maheed Abrar: "For our future roadmap, we have laid architectural hooks for real-time 2-player local co-op, procedural level generation with randomized trap seeds, hardware-accelerated OpenGL shader lighting, and dynamic boss encounters."

Ovijit Sharma: "The entire project compiles cleanly and runs at a rock-solid 60 FPS in Visual Studio 2013 and modern Windows systems using native C++ and iGraphics. We now invite the honorable evaluation committee to witness our live gameplay demonstration, test the puzzles firsthand, and we welcome your questions. Thank you very much!"

SLIDE VISUALS

Delivered objectives checklist (100% complete), 4 future roadmap pillars, live demo prompt button.

PRESENTER ACTIONS

LIVE DEMO Switch window to `demo.exe` / Visual Studio.

INVITE Invite teacher to test Floor 4 / Floor 5 puzzles or kick a penalty!

Live Demonstration Playbook — Step-by-Step Defense Sequence

Step	Feature Demonstrated	Lead Presenter	Key Visual / Technical Action in Game
1. Cold Open	Game Intro & Vignette	Ovijit Sharma	Launch <code>demo.exe</code> . Show dark vignette, poetic narrative text, press Space to enter main menu.
2. 3D Menus	World Carousel & Heroes	Nabeel Saad Borno	Navigate Chapter 1–4 cards in 3D carousel. Open Hero Select; showcase RenoSir and CR7 skins.
3. Floor 1 & 2	Platforming & Traps	Shahriar Rythm	Play Floor 1. Jump across spikes, pull ceiling chain, step on 3D red button to lower drawbridge.
4. Floor 4 & 5	Tricky Puzzle Logic	Shahriar Rythm	Showcase Floor 4 brute-force door push (no key) and Floor 5 mid-air key catch before floor collapses.
5. Shootout	Penalty Derby vs Messi	Maheed Abrar	Open 7-Day Login calendar. Trigger Daily Shootout. Aim crosshair, charge power gauge, score SIUUU goal!
6. Persistence	Save Data Verification	Maheed Abrar	Close game, show <code>tricky_castle_save.dat</code> timestamp, re-open game to prove progress and stars saved.

PRESENTER POCKET CHEAT SHEETS (CUE CARDS)

Quick 1-Page Summary for Handheld Cue Cards During the Live Defense

👑 OVIJIT SHARMA • 0:00 – 3:00 (Slides 1 – 6)

- **Slide 1 (Intro):** Welcome faculty. Replicate poetic intro from `iMain.cpp`. Moody vignette & golden prompt.
- **Slide 2 (Roster):** Introduce 4 teammates, 3 mins each, 100% C++ proposal scope delivery.
- **Slide 3 (Physics):** 14-state FSM. 60 FPS Euler integration ($y = y_0 + v_y * t - 0.5 * g * t^2$). Dual gravity (+g/-g ceiling walk). AABB tile collision.
- **Slide 4 (Animation):** Procedural deformation. Idle sway $\sin(t * 0.045) * 0.028$. Run bounce $|\sin(t * 0.20)| * 4.0$. Squash/stretch (0.90/1.12 jump, 1.18/0.82 land).
- **Slide 5 (Graphics/Input):** `drawGlowingText` concentric alpha passes. Vignette shader. Unified input in `InputHandler.h`.
- **Slide 6 (RenoSir Collab):** AI Strategist lore, Speed 4.0, Jump 12.0. `renoSirOneLevelTrial = true`. Bridge to Day 7 unlock.
- **Handover:** Hand over to Shahriar Rythm for Level Architecture.

🏰 SHAHRIAR RYTHM • 3:00 – 6:00 (Slides 7 – 10)

- **Slide 7 (Catalog):** 24 floors across 4 chapters. Chapter 1 (Princess), Ch 2 (Citadel), Ch 3 (Witch), Ch 4 (Dragon). `LevelInfo` struct.
- **Slide 8 (Traps):** `WorldObjects.h`. 3D red button sinks 8px. Procedural flickering torches. Ceiling pull-chain lowers drawbridge. 3 door types.
- **Slide 9 (Puzzles):** Gravity reverse pads. Spring pads (2.5x jump). Pushable crates. Expectation subversion: Floor 4 shove door; Floor 5 catch key mid-air.
- **Slide 10 (Spawner):** `loadLevel(int lvl)` clean reset without memory leaks. Zero load latency on restart. Two-tier hint and instant solution dispenser.
- **Handover:** Hand over to Nabeel Saad Borno for UI and menus.

🎨 NABEEL SAAD BORNO • 6:00 – 9:00 (Slides 11 – 14)

- **Slide 11 (Vector UI):** `MenuScreens.h` (1,363 lines). `draw3DButton` with highlight bevel & click offset. Top HUD telemetry & color tokens.
- **Slide 12 (3D Carousel):** Pseudo-3D depth perspective scaling. Center card 100%, peripheral cards 82% scale with 0.65 opacity. 6-champion costume carousel.
- **Slide 13 (Maps & Stars):** 12-floor grid map per chapter. 3-star rating engine (time, zero deaths, secret coin). Procedural polar star rendering. Juventus CR7 unlock.
- **Slide 14 (Modals & Toast):** Non-blocking `showToast` queue with ease-in/out alpha fade. RenoSir Collab modal card. Full settings menu.
- **Handover:** Hand over to Maheed Abrar for systems & mini-game.

🌐 MAHEED ABRAR • 9:00 – 12:00 (Slides 15 – 18)

- **Slide 15 (Persistence/Audio):** `SaveSystem.h` binary serialization to `tricky_castle_save.dat`. Corruption fallback. Windows MCI (`mciSendString`) loop BGM.
- **Slide 16 (Daily Treasures):** 7-day calendar. Day 7 = RenoSir unlock! Reward claimed via Penalty Shootout. Score to claim & SIUUU; miss resets streak!
- **Slide 17 (Penalty Derby):** `PenaltyGame.h` (1,594 lines). Power gauge. 3D ball radius scaling 24px to 9px. Messi AI goalkeeper diving & net ripple physics.
- **Slide 18 (Diagnostics/QA):** `--autocapture` headless CLI test across 14 states. `saveGUIScreenshot` via `glReadPixels` to BMP. Memory leak audit.
- **Slide 19 (Wrap-up):** Joint conclusion. 100% proposal delivery. Launch live gameplay demonstration.